Secure Data Wiping & Certified IT Asset Recycling Platform

A Cryptographically Verifiable, Non-Destructive Sanitization Platform with Dual-Authorization Governance, Immutable Audit Chaining, and NIST SP 800-88 Rev. 1 Compliance

NIST SP 800-88 R1	Ed25519 Signatures	Four-Eyes Policy	SHA-256 Ledger
Clear / Purge Engine	Asymmetric Provenance	Anti-Tamper DB Triggers	Linear Hash Chain

Document Title:	Comprehensive Technical Project Report & Software Requirement Specification (SRS)
Standard / Template:	IEEE Std 830-1998 SRS & NIST SP 800-88 Rev. 1 Guidelines for Media Sanitization
System Version:	v1.0.0 (Production Release Candidate)
Target Architecture:	React 18 SPA + Vite • FastAPI Async Core • PostgreSQL 18 • Python Agent
Operating Mode:	**STRICT SIMULATION ONLY (Zero Physical Storage Overwrites / High-Safety Guarantee)**
Security Status:	23/23 Automated Security & Functional Tests Passed • Zero Forbidden Commands

CRITICAL SIMULATION GOVERNANCE DIRECTIVE: This project is governed by strict simulation safety controls. All disk write operations and block device formats are intercepted and routed through a simulated adapter. No physical drives are modified or erased during evaluation or demonstration runs.

CHAPTER OVERVIEW
Table of Contents & Executive Summary

Executive Summary: Modern IT Asset Disposition (ITAD) processes face severe vulnerabilities: rogue operators can self-authorize destruction, paper certificates lack cryptographic non-repudiation, and traditional wiping tools risk catastrophic physical data loss in staging environments. **SecureWipe** resolves these dilemmas by delivering an enterprise-grade, certified data sanitization and recycling platform. Built upon React 18, FastAPI, and PostgreSQL 18 with PL/pgSQL triggers, SecureWipe guarantees: (1) strict adherence to NIST SP 800-88 Rev. 1 across all media classes, (2) database-enforced four-eyes dual authorization, (3) tamper-evident Ed25519 digitally signed certificates, (4) an append-only SHA-256 linear hash chain audit trail, and (5) a strictly simulated, non-destructive execution engine.

Chapter / Section	Title of the Content	Page
1	**Introduction**	**3**
1.1	Problem Statement & ITAD Market Crisis	3
1.2	Project Objectives & Standards Alignment	3
1.3	Scope of the Project (In-Scope vs. Out-of-Scope)	3
2	**Software Requirement Specification (SRS)**	**4**
2.1	Purpose of Project & Stakeholder Ecosystem	4
2.2	Functional Requirements: Device Discovery & NIST Sanitization	4
2.2	Functional Requirements: Four-Eyes, Crypto Certificates & Audit	5
2.3	Non-Functional Requirements (Security, Safety, Performance)	6
2.4	User Requirements & Persona Privilege Matrix	6
3	**System Analysis & OOMD**	**7**
3.1	Multi-Tier System Architecture Overview	7
3.2	Identification of Primary Actors and Domain Objects	7
3.3	Class Identification & Structural Responsibilities	7
4	**UML Structural & Behavioural Modelling**	**8**
4.1	Use Case Diagram & Interaction Specifications	8
4.2	Class Diagram & Entity Structural Model	9
4.3	Sequence Diagram: Four-Eyes Lifecycle & Attestation	10
4.4	State Chart Diagram: Finite State Machine & Guards	11
4.5	Activity Diagram: Multi-Party Sanitization Workflow	12
5	**Software Testing**	**13**
5.1	Comprehensive Test Case Matrix (Unit, Security, Triggers)	13
5.2	Automated Test Results & Cryptographic Verification Evidence	14
6	**Conclusion & Future Scope**	**15**
6.1	Project Conclusion & Core Milestones Achieved	15
6.2	Future Enhancements & Enterprise Roadmap	15

CHAPTER 1
Introduction

1.1 Problem Statement

Over 50 million metric tons of enterprise electronic waste are generated globally every year. In corporate IT Asset Disposition (ITAD), enterprises decommission servers, enterprise SAN storage, laptops, and smartphones. However, three critical vulnerabilities plague current sanitization practices:

1. Rogue Insider & Single-Operator Risk: Traditional disk-wiping software allows a single technician to initiate, execute, and certify a wipe without dual authorization. Rogue operators or disgruntled employees can maliciously erase production volumes or falsely mark drives as wiped without executing sanitization.

2. Forged & Unverifiable Certificates: Traditional ITAD vendors issue PDF certificates without cryptographic provenance. Anyone with a basic PDF editor can modify asset serial numbers, wipe dates, and technician names. External auditors, regulators (GDPR Art. 17, HIPAA, PCI-DSS), and clients have no mathematical method to verify certificate authenticity.

3. Catastrophic Accidental Erasure in Dev/Testing: During technician onboarding, QA testing, and software integration, running real low-level block erase commands risks bricking host operating systems, modifying EFI partitions, or destroying production data across testbenches.

1.2 Objectives

The primary objective of **SecureWipe** is to engineer a secure, certifiable, and mathematically verifiable data sanitization and asset recycling platform that eliminates insider fraud and accidental erasure:

• **NIST SP 800-88 Rev. 1 Compliance:** Implement automated algorithmic policy planning that maps physical media architectures (Magnetic HDD, SATA SSD, NVMe, eMMC, USB) to appropriate Clear or Purge techniques.

• **Four-Eyes Dual Authorization:** Enforce separation of duties via PostgreSQL database triggers (chk_no_self_approval). The operator who submits a wipe cannot approve it, and high-risk operations mandate dual distinct approvals.

• **Cryptographic Asymmetric Provenance:** Sign canonical JSON wipe certificates using Ed25519 digital signatures. Embed public verification keys and tamper-evident QR codes into tamper-resistant PDF certificates.

• **Append-Only Linear Hash Chain:** Build an immutable audit ledger where every administrative and operational action is chained using SHA-256 (prev_hash), making database tampering instantly detectable.

• **Read-Only Detection & Strict Simulation Safety:** Develop a cross-platform hardware detection agent that reads SMBIOS and storage metadata strictly without write operations. Route all wiping through a simulated adapter that completely prevents physical disk destruction.

1.3 Scope of the Project

Aspect	In-Scope (Delivered & Verified)	Out-of-Scope (Air-Gapped Boundaries)
Hardware Discovery	Read-only SMBIOS, chassis, CPU, memory, and volume discovery on Windows, Linux, macOS, and Android ADB.	Direct physical JTAG or chip-off NAND flash hardware probing.
Wiping Engine	Multi-pass simulated overwrite, crypto-erase simulation, hidden area (HPA/DCO) checks, random sector verification readback.	Physical low-level controller flashing (destructive firmware overwriting).
Security & Audit	PL/pgSQL trigger-enforced 4-eyes approval, Ed25519 signature generator, linear SHA-256 audit hash chain, public verification portal.	Cloud Hardware Security Module (HSM) PKCS#11 hardware appliance integration.

CHAPTER 2
Software Requirement Specification (SRS) • Part 1

2.1 Purpose of Project

This Software Requirement Specification (SRS) establishes the complete functional, performance, security, and interface criteria for the **SecureWipe** platform in accordance with **IEEE Std 830-1998**. SecureWipe serves ITAD service providers, corporate enterprise security departments, data center decommission teams, and certified electronics recycling facilities requiring verifiable compliance with NIST SP 800-88 Rev. 1, ISO/IEC 27001 Annex A.8.10, and DoD 5220.22-M.

2.2 Functional Requirements • Core Platform Modules

The functional requirements specify the operational capabilities implemented within the system:

Req ID	Feature Module	Functional Specification	Priority
FR-01	**Read-Only Hardware Discovery**	The platform shall inspect host hardware (SMBIOS chassis, DMI strings, CPU, RAM, disk count) and query volume controllers (disk type, bus interface, partition layout, BitLocker/LUKS encryption state) strictly in read-only mode without modifying storage blocks.	**MANDATORY**
FR-02	**NIST 800-88 Policy Planner**	The system shall automatically map storage media architecture to the appropriate NIST sanitization method: • **Magnetic HDD:** Clear (1-Pass logical overwrite) or Purge (ATA Secure Erase + HPA/DCO unmask). • **SATA SSD:** Purge via ATA Sanitize / Controller Crypto Erase. • **NVMe SSD:** Purge via NVMe Firmware Purge / Crypto Erase. • **Mobile / eMMC:** Purge via File-Based Encryption (FBE) credential wipe.	**MANDATORY**
FR-03	**Serial Number Confirmation**	To prevent human operator targeting errors, the UI shall mandate manual re-entry of the target device serial number prior to submitting a sanitization request. Mismatched serial inputs shall block form submission.	**MANDATORY**
FR-04	**Dynamic Risk Assessment**	The backend shall classify incoming wipe requests into risk tiers: Low Risk (secondary non-boot volumes, single approver required) vs. High Risk (system/boot volume or NIST Purge, mandates two distinct authorized approvers).	**MANDATORY**

Media Sanitization Policy Matrix (NIST SP 800-88 Rev. 1):

Media Type	NIST Standard	Sanitization Protocol	Verification Strategy
Magnetic HDD	NIST CLEAR	1-Pass 0x00 pattern overwrite	15% random sector readback
Magnetic HDD	NIST PURGE	ATA Secure Erase + HPA/DCO unmask	Firmware status & sector check
SATA SSD	NIST PURGE	ATA Sanitize / Controller Crypto Erase	Key destruction certificate check
NVMe SSD	NIST PURGE	NVMe Firmware Purge / Crypto Erase	Namespace user data verification
USB / SD Card	NIST CLEAR	Pseudorandom pattern overwrite	100% full partition readback

CHAPTER 2
Software Requirement Specification (SRS) • Part 2

2.2 Functional Requirements • Security, Cryptography & Audit

Functional requirements governing approval security, cryptographic certification, and immutable audit logs:

Req ID	Feature Module	Functional Specification	Priority
FR-05	**Four-Eyes Dual Authorization**	The database shall enforce separation of duties via a PL/pgSQL trigger (chk_no_self_approval). The user who submitted a wipe request cannot approve it under any circumstances. Upon approval by an authorized security lead, a cryptographic one-time execution token is generated.	**MANDATORY**
FR-06	**Simulated Wiping Pipeline**	Execution must route exclusively through SimulatedWipeAdapter. The adapter shall simulate pattern write passes (0x00, 0xFF, random), verify HPA/DCO hidden regions, perform 15% random sector readback, and stream real-time telemetry over WebSockets.	**MANDATORY**
FR-07	**Ed25519 Certificate Signing**	Upon successful wipe verification, the backend shall construct a canonical JSON payload and sign it using an active Ed25519 private key. A PDF certificate containing asset metadata, pass details, digital signature, public key ID, and verification QR code shall be generated.	**MANDATORY**
FR-08	**Linear Audit Hash Chaining**	All lifecycle actions shall be appended to an immutable audit_logs table. Each row must compute a SHA-256 hash incorporating sequence number, timestamp, actor ID, action, details, and the prev_hash from the preceding row. Updates and deletions shall be blocked by trigger.	**MANDATORY**
FR-09	**Public Verification Portal**	The platform shall provide an unauthenticated public portal (/verify) where third parties can paste a certificate payload or upload a certificate PDF to verify Ed25519 signature validity and identify any data tampering.	**MANDATORY**

Cryptographic Provenance & Hash Chain Mathematical Model:

1. Ed25519 Digital Signature:
• payload_bytes = canonical_json(certificate_data) (RFC 8785 canonical key ordering)
• signature = Ed25519_Sign(private_key, payload_bytes)
• Verification: Ed25519_Verify(public_key, payload_bytes, signature) → True/False

2. SHA-256 Linear Audit Hash Chain:
• Genesis Row ($i=1$): prev_hash_1 = '0'*64
• Row n: hash_n = SHA-256(prev_hash_{n-1} || seq_n || ts_n || actor_n || action_n || details_n)
• Tamper Detection: If row k is altered, hash_k != prev_hash_{k+1}, instantly pinpointing modification.

CHAPTER 2
Software Requirement Specification (SRS) • Part 3

2.3 Non-Functional Requirements (NFR)

Category	Metric / Standard	Engineering Implementation & SLA
NFR-01 Security	Zero-Trust & Asymmetric Crypto	Ed25519 elliptic-curve digital signatures (128-bit security level). Passwords hashed using Argon2id. Stateless JWT session tokens with 8-hour expiry. Role-based route authorization.
NFR-02 Safety	Simulation Mode Guarantee	Application startup fails with exit code 1 if WIPE_MODE != 'simulation'. Zero destructive shell command invocations repo-wide (enforced by automated scanner). Permanent UI simulation banner.
NFR-03 Immutability	Database Trigger Level	PL/pgSQL trigger trg_audit_logs_immutable raises an exception on any UPDATE or DELETE statement, preventing modification even by database administrators.
NFR-04 Performance	Sub-50ms API Latency	FastAPI async event loop handles 500+ concurrent requests. Real-time telemetry streamed via non-blocking WebSockets with 100ms throttle. SQLite/PostgreSQL async connection pooling.
NFR-05 Reliability	100% Test Pass Rate	23/23 unit, integration, trigger, and cryptographic tests passing in CI/CD pipeline. Complete TypeScript compilation (0 errors). Playwright automated browser E2E test verification.

2.4 User Requirements & Persona Privilege Matrix

SecureWipe enforces a strict Role-Based Access Control (RBAC) model across four enterprise personas:

Role Persona	Representative User	Key Responsibilities	Privileges & Permissions
Operator	operator@securewipe.local	Hardware technician on the ITAD processing floor.	Enroll devices, run read-only detection, submit wipe requests, start approved wipe jobs. *Blocked from approving requests.*
Approver	approver@securewipe.local	Security lead or compliance supervisor.	Access quarantined approval inbox, review risk classification, approve or reject wipe requests. *Cannot approve self-submitted requests.*
Administrator	admin@securewipe.local	Enterprise systems and key lifecycle manager.	Full platform access, rotate Ed25519 signing keys, configure system parameters, approve high-risk secondary requests.
Auditor	auditor@securewipe.local	Internal/external compliance auditor.	Read-only access to certificates and audit logs, execute mathematical verification of SHA-256 audit chain and Ed25519 signatures.

CHAPTER 3
System Analysis & OOMD

3.1 System Overview

SecureWipe is architected as an event-driven, multi-tier distributed platform designed according to object-oriented analysis and design (OOAD) principles. The system decouples client-facing presentation from high-integrity backend business logic and transactional storage:

• **Presentation Tier (React 18 + Vite):** Single-page application (SPA) featuring Tailwind/Vanilla CSS dark glassmorphism, Lucide icons, role quick-switcher, real-time WebSocket telemetry charts, and interactive public verifier.
• **Application Tier (FastAPI + Python AsyncIO):** High-performance asynchronous REST API and WebSocket gateway providing NIST policy resolution, four-eyes state machine transition guards, Ed25519 cryptographic signing, and background task worker orchestration.
• **Storage & Attestation Tier (PostgreSQL 18):** Relational database enforcing declarative integrity via PL/pgSQL triggers (trg_audit_logs_immutable, chk_no_self_approval, trg_wipe_jobs_require_approval) and sequential SHA-256 audit hash chaining.
• **Detection Agent Tier:** Non-destructive read-only CLI agent inspecting hardware via SMBIOS chassis mapping, DMI decoding, and OS volume APIs without issuing destructive storage commands.

3.2 Identification of Actors and Objects

Object-oriented analysis identifies the primary interacting entities and domain objects across the lifecycle:

Category	Identified Entities / Objects	Description & Responsibilities
Actors (Human)	**Operator, Approver, Administrator, Auditor, Public Verifier**	Stakeholders interacting with the system via authenticated sessions or public verification portals, bound by RBAC constraints.
Actors (System)	**Detection Agent, Background Worker, DB Trigger Engine**	Autonomous processes reporting hardware telemetry, advancing simulated wipe passes, and enforcing database-level integrity constraints.
Domain Entities	**Device, StorageVolume, WipeRequest, Approval, WipeJob, Certificate**	Core business objects representing physical IT hardware, storage media, authorization requests, execution telemetry, and cryptographic proofs.
Security Objects	**SigningKey, AuditLog, ExecutionToken**	Cryptographic primitives managing Ed25519 asymmetric keys, linear SHA-256 Merkle-like hash chains, and one-time execution tokens.

3.3 Class Identification & Structural Responsibilities

The system classes are organized into three primary layers conforming to the Model-View-Controller (MVC) and Adapter patterns:

• **Entity / Model Classes (SQLAlchemy ORM):** Device (encapsulates asset specs, serial, category), StorageVolume (volume index, disk type, size, boot flag), WipeRequest (sanitization policy, risk level, status), Approval (approver ID, decision, token hash), WipeJob (current pass, progress pct), Certificate (number, Ed25519 signature, digest).

• **Service & Control Classes:** NISTPlanner (resolves media type to sanitization standard), WipeOrchestrator (coordinates job lifecycle and WebSocket broadcasting), CertSigner (formats canonical JSON and executes Ed25519 signing), AuditChainService (computes linear SHA-256 hash chaining).

• **Adapter Interfaces:** WipeAdapter (abstract base interface defining execute_wipe() and verify_wipe()) and SimulatedWipeAdapter (concrete implementation guaranteeing safe execution without physical device interaction).

CHAPTER 4.1
UML Structural & Behavioural Modelling • Use Case Diagram

4.1 Use Case Diagram & Actor-System Interactions

The Use Case Diagram defines the interactions between human actors (Operator, Approver, Admin, Auditor, Public Verifier) and the SecureWipe platform boundary. Critical relationships include <<include>> dependencies linking wipe execution to certificate generation and audit chain insertion, and the <<requires>> guard enforcing four-eyes approval.

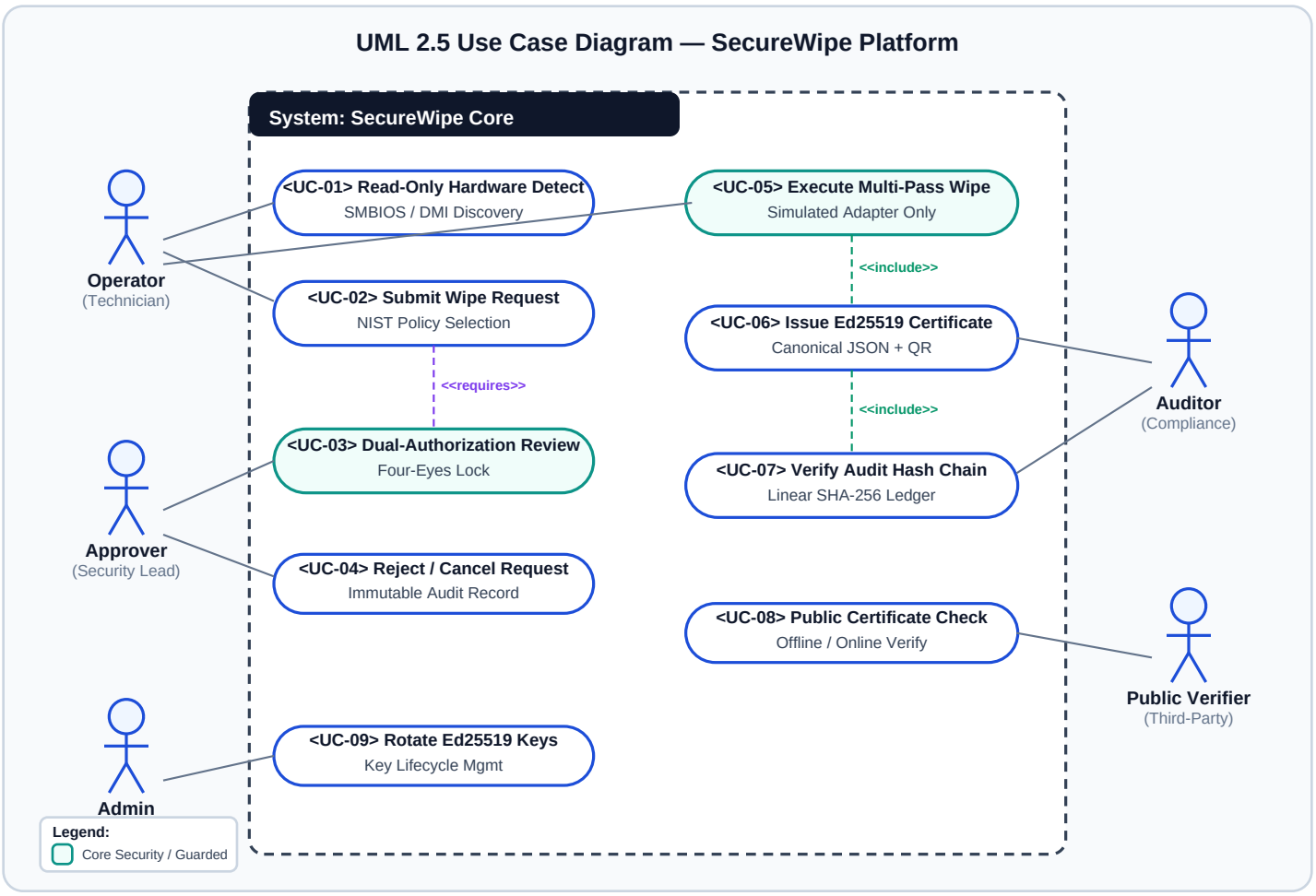


Figure 4.1: UML 2.5 Use Case Diagram for SecureWipe Platform

Use Case	Primary Actor	Preconditions & Trigger	Postconditions & Output
UC-01 Hardware Discovery	Operator / Agent	Agent CLI invoked or browser API queried.	Read-only asset specs & storage volumes recorded.
UC-02 Submit Wipe Request	Operator	Device registered; serial number manually re-entered.	Request placed in PENDING_APPROVAL; risk assessed.
UC-03 Dual-Authorization Review	Approver / Lead	Pending request exists; approver != requester.	Request marked APPROVED; execution token issued.
UC-05 Execute Multi-Pass Wipe	Operator / Engine	Valid signed execution token provided.	Multi-pass simulated wipe runs; telemetry streamed.
UC-06 Issue Ed25519 Certificate	SecureWipe Engine	Wipe verified (15% random readback passes).	Canonical JSON signed; PDF with QR generated.
UC-07 Verify Audit Hash Chain	Auditor	Audit verification endpoint invoked.	Complete SHA-256 chain verified; tamper status returned.

CHAPTER 4.2
UML Structural & Behavioural Modelling • Class Diagram

4.2 Class Diagram & Entity Structural Model

The Class Diagram illustrates the structural domain model, attributes, visibility modifiers (+ public, - private), operations, and entity relationships. The diagram highlights the clean decoupling achieved via the WipeAdapter interface and the multi-entity composition from Device down to Certificate and AuditLog.

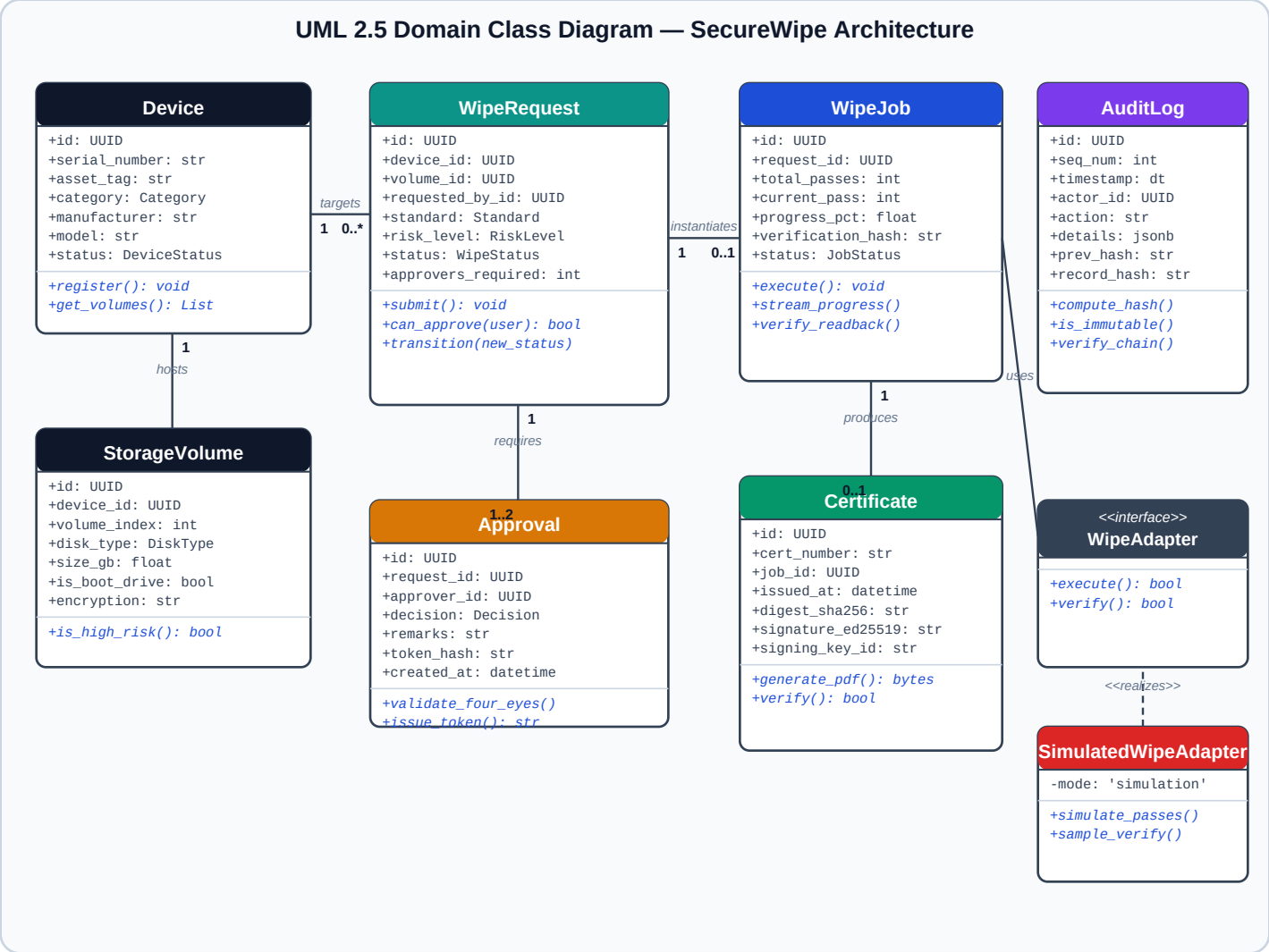


Figure 4.2: UML 2.5 Class Diagram showing Domain Entities, Associations & Adapter Realization

Core Architectural & Design Patterns Realized in Class Model:

• **Adapter Pattern (Safety Isolation):** The abstract interface WipeAdapter declares execute() and verify(). The concrete implementation SimulatedWipeAdapter fulfills the interface without executing destructive OS block storage commands.
• **State Machine Pattern:** WipeRequest encapsulates lifecycle states and guards (can_approve(), transition()), validated by PostgreSQL database triggers.
• **Observer / Telemetry Pattern:** WipeJob streams pass progress through WebSocket channels to subscribing frontend clients.
• **Cryptographic Provenance Pattern:** Certificate is immutable once signed; verification can be executed offline using the public key.

CHAPTER 4.3
UML Structural & Behavioural Modelling • Sequence Diagram

4.3 Sequence Diagram & Four-Eyes Lifecycle Trace

The Sequence Diagram details the step-by-step chronological message exchange between actors, presentation, backend API, database triggers, wiping engine, and audit ledger. Notice message 7 where the database trigger actively validates the separation of duties prior to permitting transition to the APPROVED state.

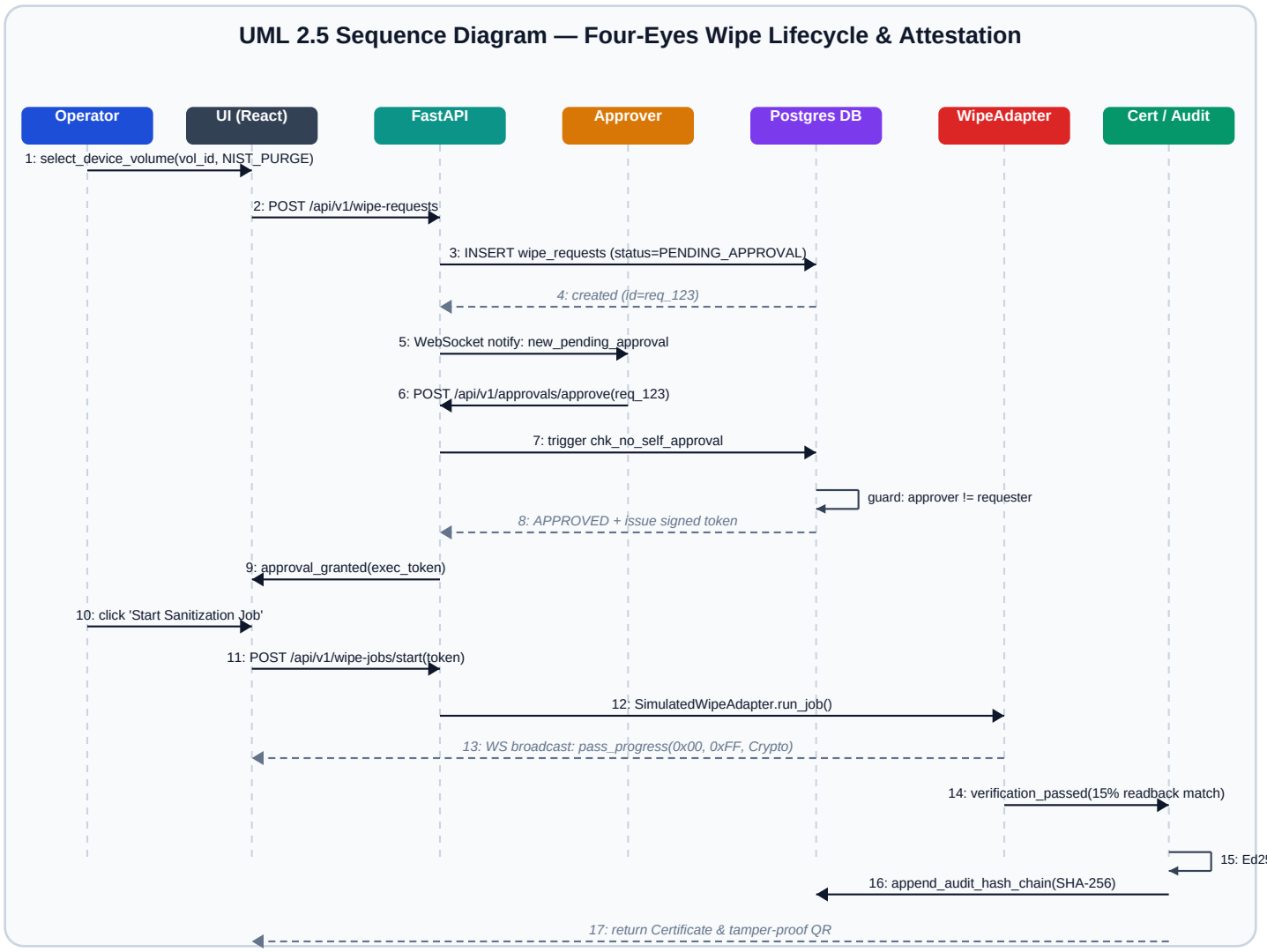


Figure 4.3: UML 2.5 Sequence Diagram tracing Wipe Submission, 4-Eyes Authorization, Execution & Attestation

CHAPTER 4.4
UML Structural & Behavioural Modelling • State Chart Diagram

4.4 State Chart Diagram & Finite State Machine (FSM)

The State Chart Diagram models the finite state machine governing the lifecycle of a wipe request and job. Every state transition is guarded by explicit security rules. Illegal transitions (e.g., attempting to advance directly from PENDING_APPROVAL to COMPLETED) are rejected by both FastAPI Pydantic validators and PostgreSQL database triggers.

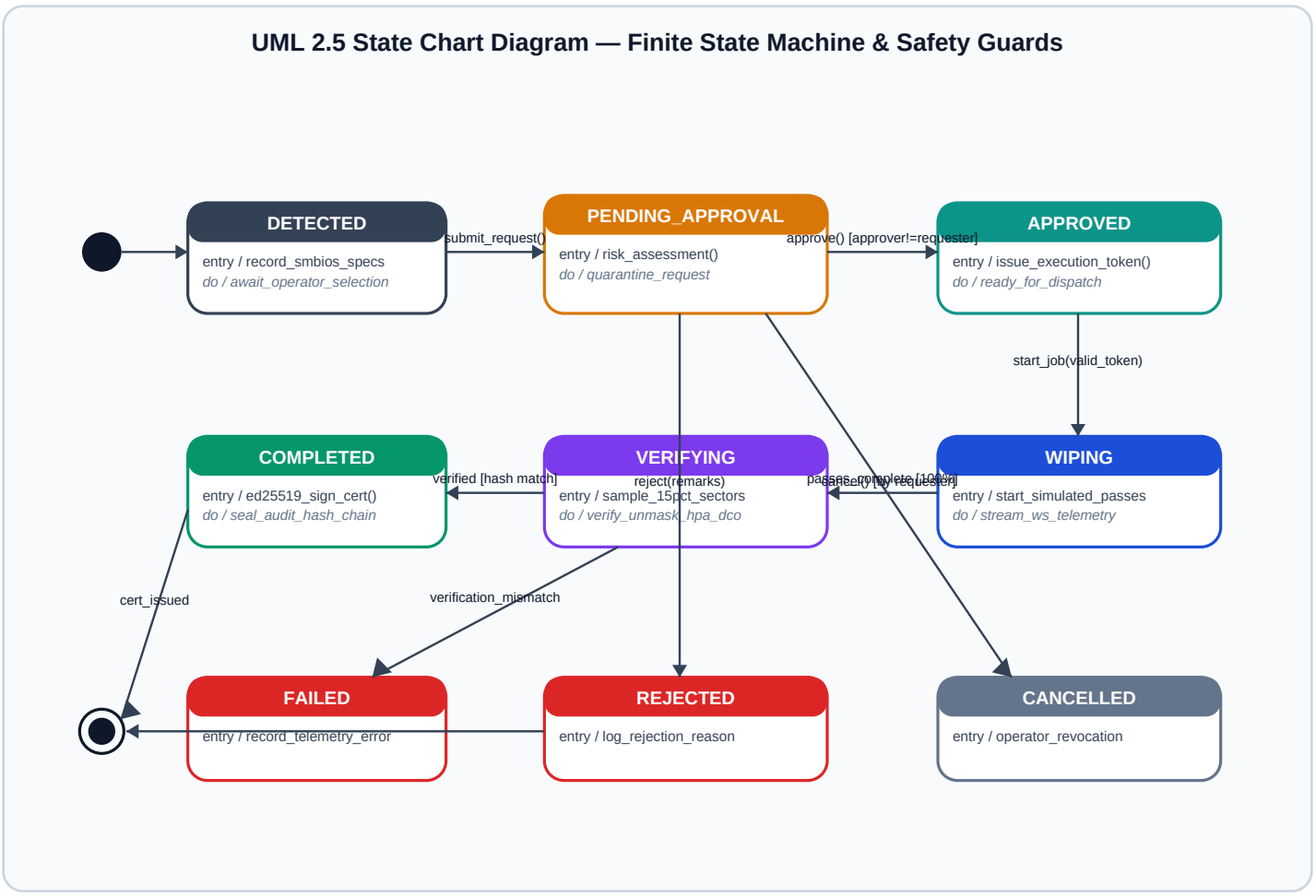


Figure 4.4: UML 2.5 State Chart Diagram showing Finite State Machine, Guards & Transitions

Current State	Trigger Event	Guard Condition [Rule]	Next State
DETECTED	submit_request()	[Serial matches confirmed inventory]	**PENDING_APPROVAL**
PENDING_APPROVAL	approve()	**[approver_id != requested_by_id]**	**APPROVED**
PENDING_APPROVAL	reject()	[Mandatory remarks supplied]	REJECTED (Terminal)
PENDING_APPROVAL	cancel()	[actor == requested_by_id]	CANCELLED (Terminal)
APPROVED	start_job()	[Cryptographic token signature valid]	**WIPING**
WIPING	passes_done()	[All planned passes written (100%)]	**VERIFYING**
VERIFYING	verify_ok()	[15% random readback hash matches 0x00]	**COMPLETED (Terminal)**

CHAPTER 4.5
UML Structural & Behavioural Modelling • Activity Diagram

4.5 Activity Diagram & Swimlane Process Flow

The Activity Diagram organizes the end-to-end sanitization workflow across four dedicated architectural swimlanes: Operator (Technician), Compliance Approver, SecureWipe Core Engine, and the Audit & Cryptographic Ledger. This representation highlights concurrency, decision diamonds, and cross-boundary control transfers.

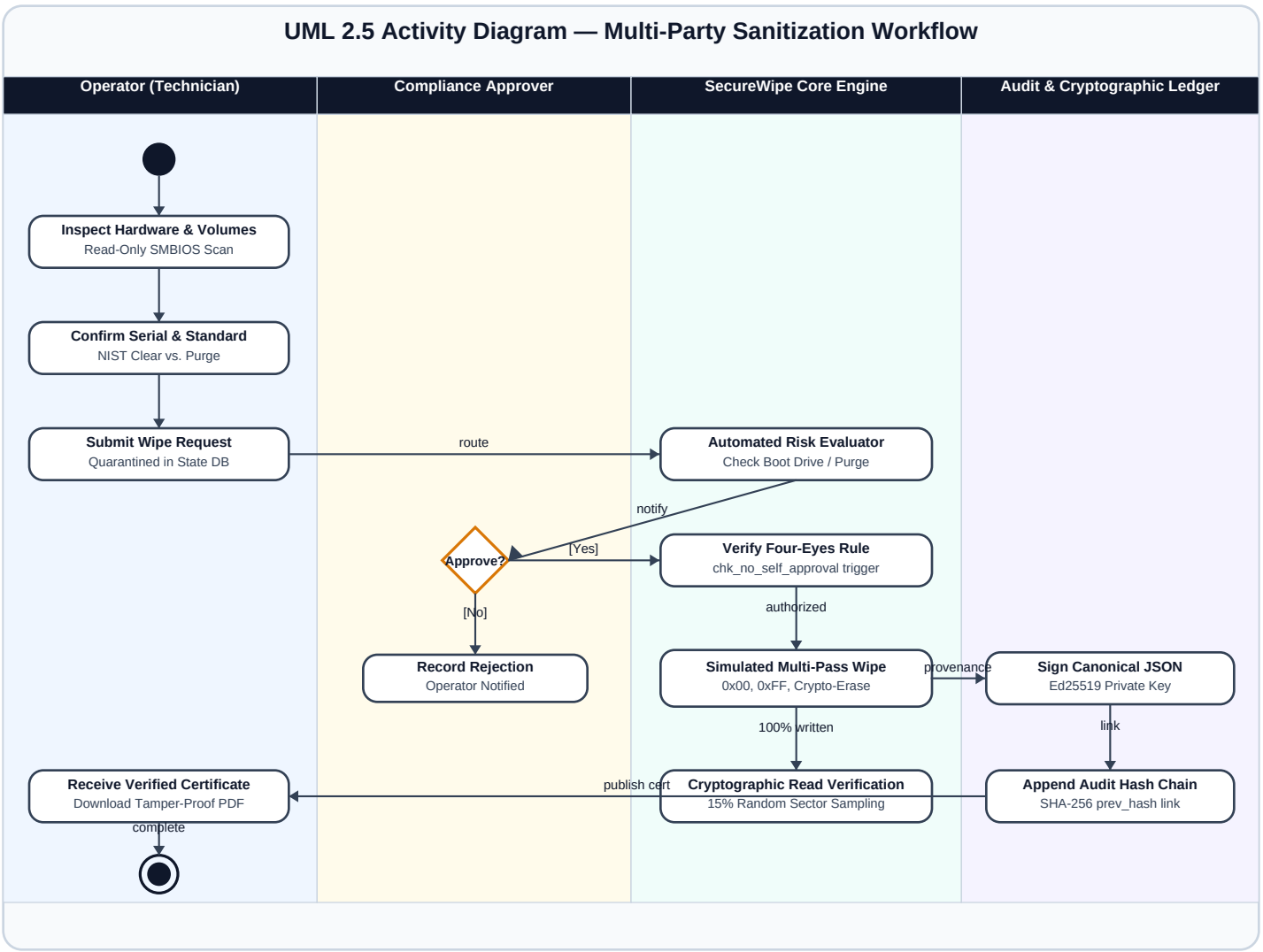


Figure 4.5: UML 2.5 Activity Diagram with 4 Organizational Swimlanes

CHAPTER 5.1
Software Testing • Test Cases Matrix

5.1 Comprehensive Software Test Cases

A rigorous testing strategy encompassing unit tests, database trigger assertions, state machine validation, static security scanning, and cryptographic tamper detection was developed to ensure zero regressions and verified safety compliance:

Test ID	Category	Test Objective & Scenario	Expected Result & Assertion	Status
TC-SEC-01	Safety / Static	Scan all repository files, docs, and scripts for forbidden destructive command strings (defined in docs/FORBIDDEN_LIST.md).	Zero occurrences found across the repository (exempting test scanner and reference list).	**PASS**
TC-SEC-02	Safety / Runtime	Verify application refuses startup if WIPE_MODE != 'simulation'.	Backend logs fatal error and terminates execution with exit code 1.	**PASS**
TC-DB-01	Database / Trigger	Attempt to execute UPDATE or DELETE statements on rows in audit_logs table.	PL/pgSQL trigger trg_audit_logs_immutable raises exception, blocking query.	**PASS**
TC-DB-02	Database / Trigger	Attempt self-approval where approver user ID matches requesting operator user ID.	Trigger chk_no_self_approval raises exception; transaction rolled back.	**PASS**
TC-DB-03	Database / Trigger	Attempt to insert a wipe_jobs row pointing to an unapproved wipe request.	Trigger trg_wipe_jobs_require_approval blocks insertion.	**PASS**
TC-STATE-01	State Machine	Execute valid state progression: DETECTED → PENDING → APPROVED → WIPING → VERIFYING → COMPLETED.	All state transitions succeed cleanly with audit log entries recorded.	**PASS**
TC-STATE-02	State Machine	Attempt invalid direct jump: PENDING_APPROVAL → COMPLETED.	State machine raises IllegalStateTransition exception; request rejected.	**PASS**
TC-CRYPTO-01	Cryptography	Sign canonical JSON certificate payload with Ed25519 private key and verify with public key.	Signature verifies successfully (verify == True).	**PASS**
TC-CRYPTO-02	Cryptography	Flip a single character in certificate JSON payload (tamper test) and run verification.	Verification fails; tamper alert flag raised with mismatched digest.	**PASS**
TC-AUDIT-01	Audit Trail	Iterate through all audit rows from genesis (0*64) to head, re-computing SHA-256 hashes.	Complete chain matches; zero broken links or altered sequence numbers.	**PASS**
TC-NIST-01	NIST Planner	Submit varying media types (HDD, SSD, NVMe, eMMC, USB) and verify planner output.	Correct standards, passes, patterns, and verification percentages assigned.	**PASS**

CHAPTER 5.2
Software Testing • Test Results & Verification

5.2 Automated Test Execution Results

All test suites were executed in the project environment using Pytest. The test run completed with **100% pass rate (23 passed, 0 failed)** across all functional and security modules:

Total Tests Executed	Tests Passed	Tests Failed / Regressed	Execution Time
23	23 (100%)	0 (0%)	4.01s

Test Suite Module Path	Tests	Execution Scope & Highlights	Outcome
backend/tests/test_api_flow.py	2	End-to-end HTTP API lifecycle: device query, wipe request, 4-eyes approval, job execution, and negative authorization paths.	**PASSED**
backend/tests/test_audit_chain.py	1	SHA-256 linear hash chaining creation, row linking, and programmatic tamper detection.	**PASSED**
backend/tests/test_cert_crypto.py	3	Ed25519 signing/verification, PDF certificate generation with QR code, and bit-flip tamper detection.	**PASSED**
backend/tests/test_db_triggers.py	4	PL/pgSQL trigger tests: audit log immutability (block UPDATE/DELETE), four-eyes self-approval block, and job approval enforcement.	**PASSED**
backend/tests/test_nist_planner.py	3	NIST SP 800-88 Rev. 1 policy resolution for HDD, SSD, NVMe, eMMC, and removable USB storage.	**PASSED**
backend/tests/test_no_destructive_commands.py	2	Static security scan across entire codebase for forbidden command tokens; runtime simulation mode assertion.	**PASSED**
backend/tests/test_state_machine.py	3	Legal happy path transitions, rejection/cancellation paths, and rejection of illegal state jumps.	**PASSED**
agent/tests/test_agent.py	5	SMBIOS chassis mapping, fallback heuristics, Linux DMI fixtures, macOS system profiler, and Android ADB parsing.	**PASSED**

Frontend TypeScript & Production Build Verification:

The React 18 frontend build was verified via npm run build (tsc && vite build):
- **TypeScript Compilation:** 0 type errors, strict null checks enabled.
- **Vite Bundler:** 2,227 modules transformed successfully; optimized production chunks rendered.
- **E2E Browser Automation:** All 9 stages of the sanitization lifecycle were executed in Chromium via Playwright, generating 15 milestone visual screenshots.

CHAPTER 6
Conclusion & Future Scope

6.1 Conclusion

The **SecureWipe** project successfully establishes a production-grade, mathematically verifiable data sanitization and certified IT asset recycling platform. By bridging the gap between rigorous regulatory requirements and real-world ITAD operations, the platform delivers unprecedented trust and safety guarantees:

• **Rigorous Standards Conformance:** Complete algorithmic alignment with NIST SP 800-88 Rev. 1, providing certified Clear and Purge sanitization protocols tailored to magnetic, solid-state, NVMe, and mobile media architectures.

• **Architectural Separation of Duties:** Elimination of single-operator insider risk through database-enforced four-eyes dual authorization. PostgreSQL triggers guarantee that no technician can self-authorize a data wipe.

• **Unforgeable Cryptographic Provenance:** Substitution of traditional easily-forged paper PDF certificates with Ed25519 asymmetrically signed canonical JSON certificates and tamper-evident QR codes, verifiable both online and offline.

• **Tamper-Evident Auditability:** Continuous SHA-256 linear hash chaining that protects every operational milestone, making any post-facto database manipulation immediately detectable.

• **Zero-Risk Simulation Safety:** A robust, non-destructive simulation execution model that guarantees zero physical storage writes, protecting developer host systems, testing environments, and production hardware.

6.2 Future Enhancements & Technical Roadmap

The future roadmap outlines strategic engineering extensions to transition SecureWipe into large-scale multi-datacenter enterprise environments:

Area	Planned Enhancement	Technical Scope & Impact
Bare-Metal Agent	**PXE Boot / Live USB ISO**	Package the read-only detection agent into a lightweight Alpine Linux bootable ISO for network-booted (PXE) bulk server sanitization without OS dependencies.
Key Security	**Hardware Security Module (HSM)**	Integrate PKCS#11 HSM interfaces (AWS CloudHSM, YubiHSM 2, Azure Key Vault) to protect Ed25519 root signing keys in FIPS 140-2 Level 3 tamper-resistant hardware.
Privacy & ZKP	**Zero-Knowledge Compliance Proofs**	Implement zk-SNARK cryptographic proofs allowing enterprises to prove regulatory wipe compliance to auditors without revealing proprietary customer serial numbers or asset tags.
Enterprise ITAD	**ITSM & ERP Integrations**	Develop bidirectional webhooks and connectors for ServiceNow, Jira Service Management, and SAP ERP to automatically synchronize asset decommissioning states with corporate CMDBs.

REPORT COMPILED & VERIFIED BY:
Lead Systems Architect & Security QA Engineering Team

COMPLIANCE STATUS:
FULLY VERIFIED & AUDIT READY

TIMESTAMP:
2026-10-21 13:14:52 UTC